INTERSECTION OF TWO ARRAYS

```
class Solution:
    def intersection(self, nums1, nums2):
        return list(set(nums1) & set(nums2))
```

INTERSECTION OF TWO ARRAYS TWO

```
class Solution:
    def intersect(self, nums1, nums2):
        count = {}
        result = []

        for x in nums1:
            count[x] = count.get(x, 0) + 1

        for x in nums2:
            if count.get(x, 0) > 0:
                result.append(x)
                count[x] -= 1

        return result
```

VALID PERFECT SQUARE

```
class Solution:
    def isPerfectSquare(self, num):
        left = 1
        right = num

        while left <= right:
            mid = (left + right) // 2

            if mid * mid == num:
                return True
            elif mid * mid < num:
                left = mid + 1
            else:
                right = mid - 1

        return False
```

GUESS NUMBER HIGHER OR LOWER

```
# The guess API is already defined.
# def guess(num: int) -> int:


class Solution:
    def guessNumber(self, n):
        left = 1
        right = n

        while left <= right:
            mid = left + (right - left) // 2

            result = guess(mid)

            if result == 0:
                return mid
            elif result == -1:
                right = mid - 1
            else:
                left = mid + 1
```

RANSOM NOTE

```
class Solution:
    def canConstruct(self, ransomNote, magazine):
        count = {}

        for ch in magazine:
            count[ch] = count.get(ch, 0) + 1

        for ch in ransomNote:
            if count.get(ch, 0) == 0:
                return False
            count[ch] -= 1

        return True
```

FIRST UNIQUE CHARACTER

```
class Solution:
    def firstUniqChar(self, s):
        count = {}

        for ch in s:
            count[ch] = count.get(ch, 0) + 1

        for i in range(len(s)):
            if count[s[i]] == 1:
                return i

        return -1
```

FIND THE DIFFERENCE

```
class Solution:
    def findTheDifference(self, s, t):
        result = 0

        for ch in s:
            result ^= ord(ch)

        for ch in t:
            result ^= ord(ch)

        return chr(result)
```

IS SUBSEQUENCE

```
class Solution:
    def isSubsequence(self, s, t):
        i = 0

        for ch in t:
            if i < len(s) and s[i] == ch:
                i += 1

        return i == len(s)
```

BINARY WATCH

```
class Solution:
    def readBinaryWatch(self, turnedOn):
        result = []

        for hour in range(12):
            for minute in range(60):
                if (bin(hour).count("1") + bin(minute).count("1")) == turnedOn:
                    result.append(str(hour) + ":" + str(minute).zfill(2))

        return result
```

SUM OF LEFT LEAVES

```
class Solution:
    def sumOfLeftLeaves(self, root):
        if not root:
            return 0

        total = 0

        if root.left:
            if not root.left.left and not root.left.right:
                total += root.left.val
            else:
                total += self.sumOfLeftLeaves(root.left)

        if root.right:
            total += self.sumOfLeftLeaves(root.right)

        return total
```

CONVERT A NUMBER TO HEXA DECIMAL

```
class Solution:
    def toHex(self, num):
        if num == 0:
            return "0"

        digits = "0123456789abcdef"
        result = ""

        num &= 0xffffffff

        while num:
            result = digits[num & 15] + result
            num >>= 4

        return result
```